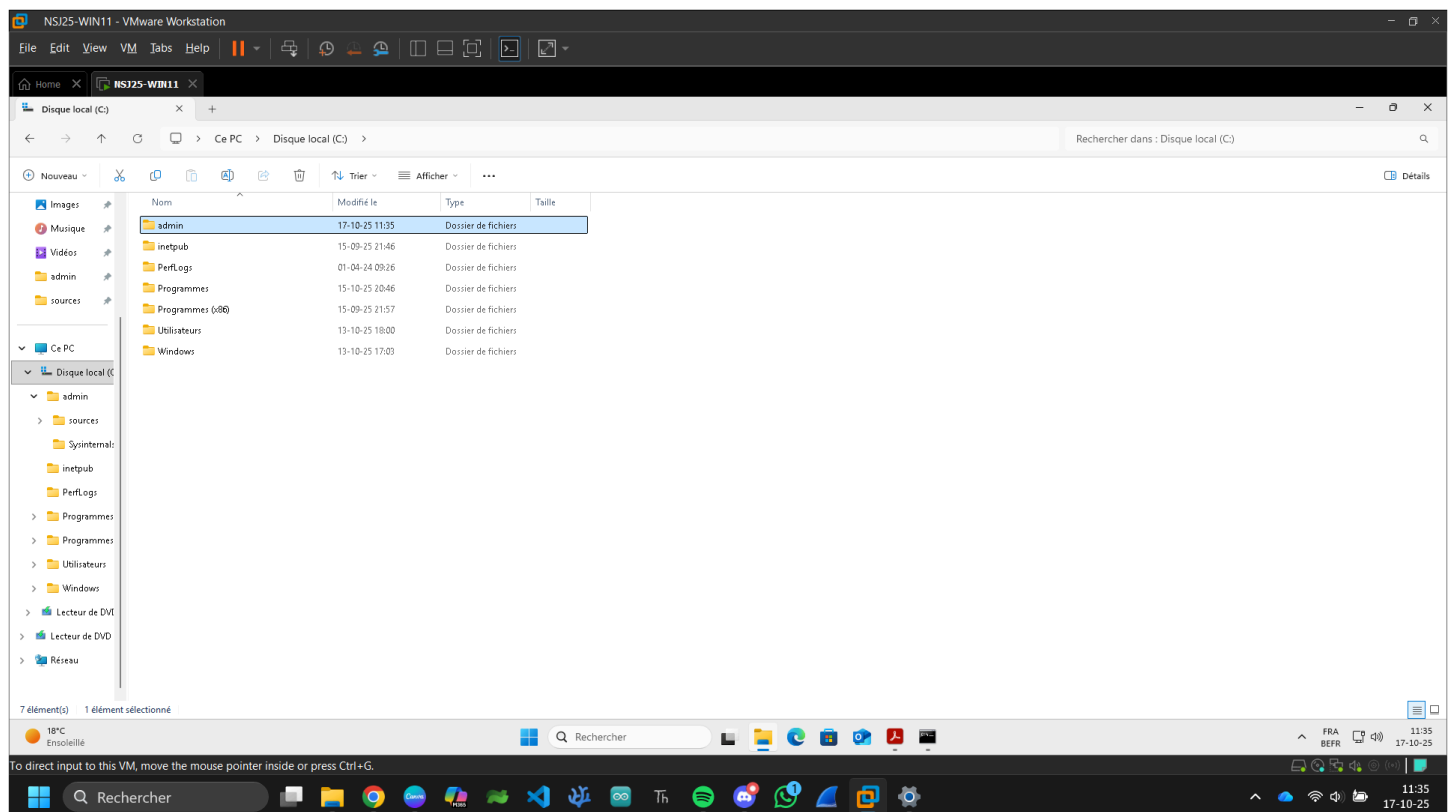
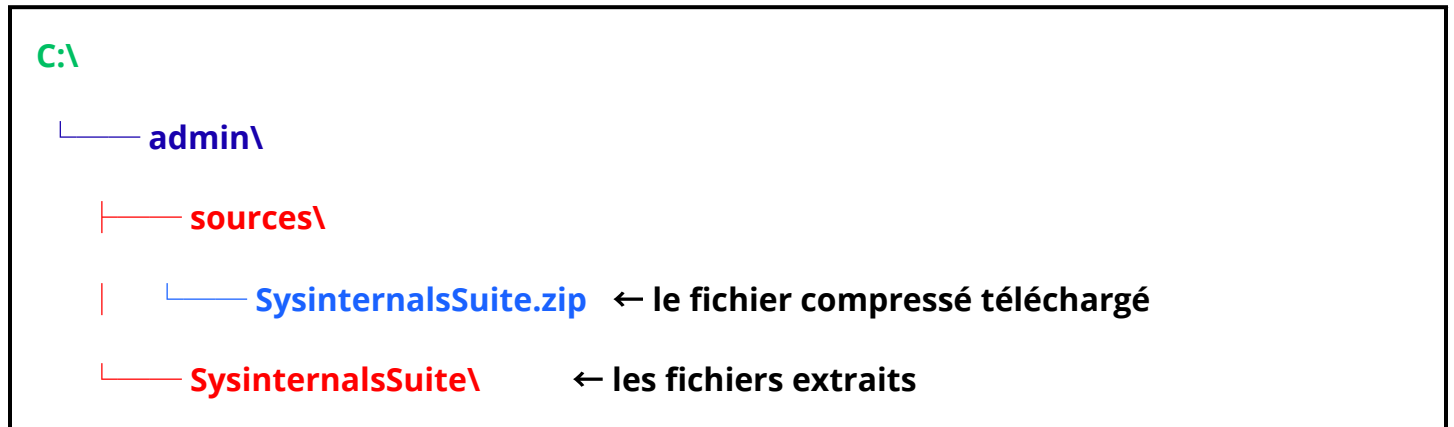


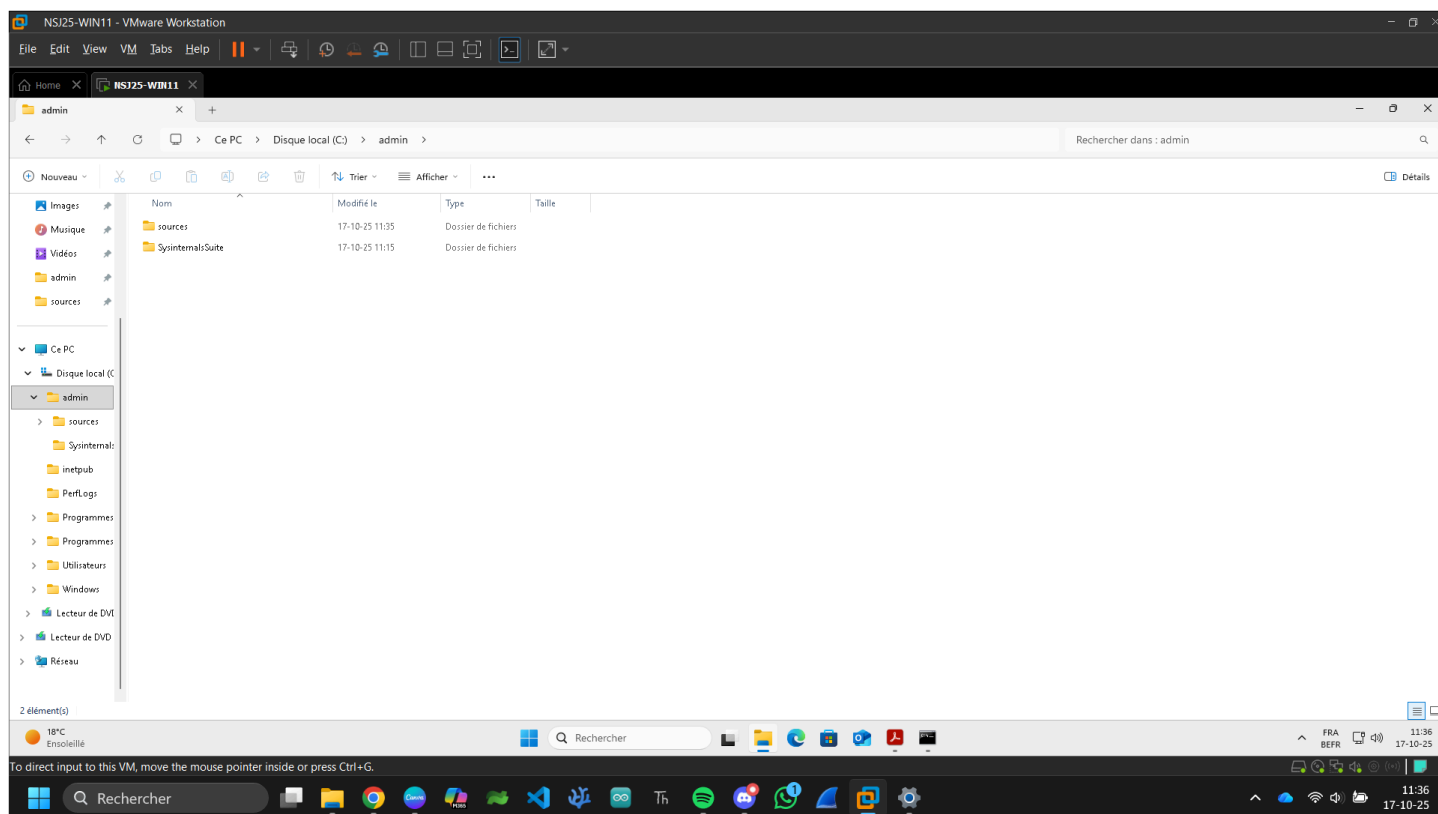
1. Créer un répertoire "admin" sur la racine du lecteur "C: \".

Ce répertoire contiendra les outils de gestions.

a) Télécharger la "sysinternals Suite" et placer le fichier compressé dans le sousrépertoire "c:\admin\sources" (créer le sous-répertoire).

b) Décompresser le contenu du fichier dans le sous-répertoire "C:\admin\SysinternalsSuite\" (créer le sous-répertoire).





<https://www.techspot.com/downloads/4680-sysinternals-suite.html>

2. Comment fonctionne le PATH de Windows ?

a) En ligne de commande, depuis le sous-répertoire "C:\admin\SysinternalsSuite\" taper la commande "psgetsid" et noter le résultat.

b) Faites de même depuis votre "home directory" (exemple : C:\Users\Laurent>).

c) Que se passe-t-il ?

Pourquoi ?

Le PATH de Windows : c'est quoi ?

Le **PATH** est une **variable d'environnement** qui contient **la liste des dossiers** où Windows va **chercher les programmes exécutables (.exe, .bat, .cmd, etc.)** quand tu tapes une commande dans le terminal (CMD ou PowerShell).

Exemple concret :

Quand tu tapes : **psgetsid**

Windows va chercher **dans chaque dossier du PATH** s'il existe un fichier psgetsid.exe.

S'il le trouve dans un des dossiers listés, il l'exécute.

S'il ne le trouve **dans aucun dossier**, il affiche : **'psgetsid' n'est pas reconnu en tant que commande interne ou externe, un programme exécutable ou un fichier de commandes.**

L'expérience demandée

a) Depuis **C:\admin\SysinternalsSuite**

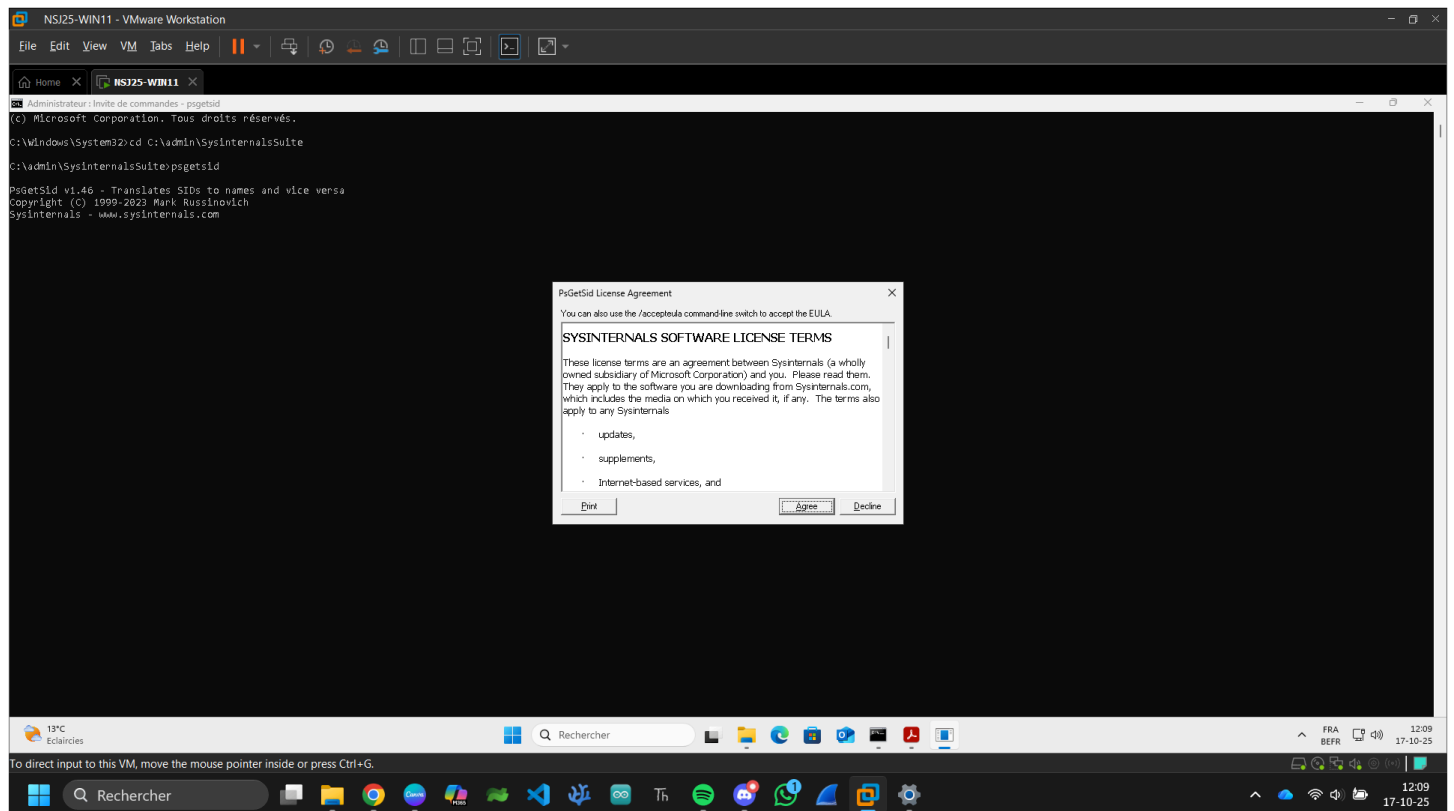
Tu ouvres un terminal ici et tu tapes :

psgetsid

➔ Le programme se trouve **dans ce répertoire**, donc **il s'exécute correctement.**

Exemple de sortie :

PsGetSid v1.44 - Translates SIDs to names and vice versa



b) Depuis ton répertoire personnel **C:\Users\<TonNom>**

Tu tapes la même commande :

psgetsid

➔ Cette fois, tu obtiens : **'psgetsid' n'est pas reconnu en tant que commande interne ou externe...**

Parce que **le dossier C:\admin\SysinternalsSuite\ n'est pas dans la variable PATH.**

Donc Windows **ne sait pas où trouver** psgetsid.exe.

Comment corriger ça

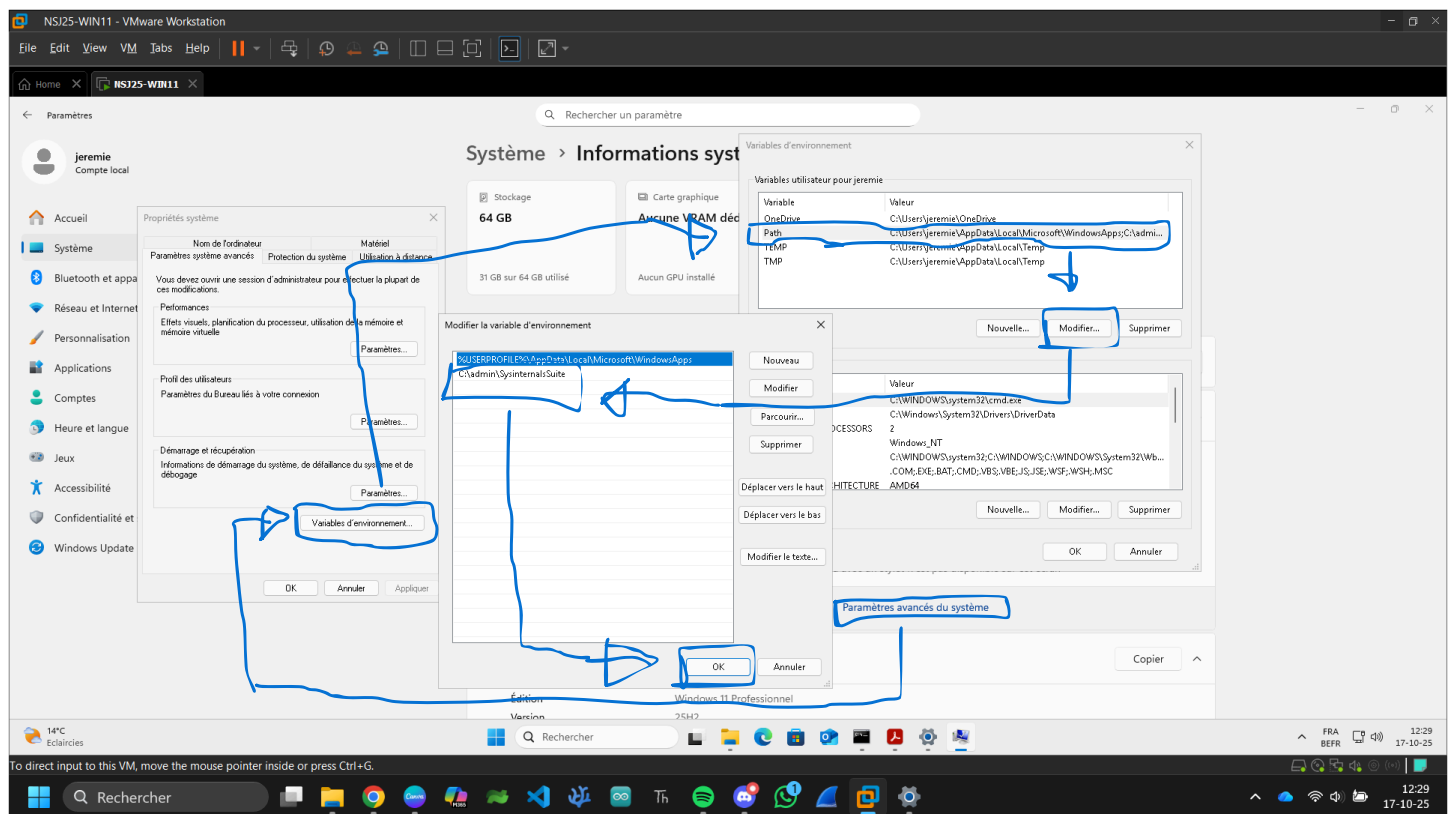
Si tu veux pouvoir exécuter psgetsid **depuis n'importe quel répertoire**, tu dois **ajouter le dossier C:\admin\SysinternalsSuite\ au PATH**.

Méthode graphique (simple)

1. Clique droit sur **Ce PC** → **Propriétés**
2. Clique sur **Paramètres système avancés**
3. Clique sur **Variables d'environnement**
4. Dans la partie du bas ("Variables système"), sélectionne Path → **Modifier**
5. Clique sur **Nouveau** → entre :
6. C:\admin\SysinternalsSuite\
7. Valide tout avec **OK**

Ensuite, test : Ouvre **un nouveau terminal** (important, pour recharger les variables) et tape : **psgetsid**

➔ Ça fonctionne **depuis n'importe où** maintenant 🎯



Que se passe-t-il si l'on réexécute le point 2 ?

Le **point 2**, c'était la commande **psgetsid** depuis un autre répertoire (par exemple ton home directory).

Avant d'ajouter le dossier au PATH

Tu obtenais :

'psgetsid' n'est pas reconnu en tant que commande interne ou externe, un programme exécutable ou un fichier de commandes.

Après avoir ajouté C:\admin\SysinternalsSuite\ dans le PATH

➡ Si tu refais la même commande :

psgetsid

Ça fonctionne désormais depuis n'importe quel répertoire !

Windows trouve **psgetsid.exe** automatiquement grâce au nouveau chemin ajouté dans la variable PATH.

Comparer le fonctionnement de PATH sous Windows et sous Linux.

La variable s'appelle **PATH** (ou Path).

Elle contient une **liste de dossiers séparés par un point-virgule ;**

Exemple :

C:\Windows\System32;C:\Windows;C:\admin\SysinternalsSuite\

Quand tu tapes une commande (notepad, ping, python, etc.),
Windows cherche l'exécutable dans :

1. le dossier courant
2. chaque dossier listé dans PATH (dans l'ordre)

Sous Linux

La variable s'appelle aussi **PATH**.

Les dossiers sont **séparés par un deux-points :**

Exemple :

/usr/local/bin:/usr/bin:/bin:/home/jeremie/scripts

Le principe est le même : le shell (bash, zsh, etc.) parcourt cette liste pour trouver un exécutable portant le nom tapé.

Différence principale :

Windows utilise ;

Linux utilise :

Linux est **sensible à la casse** (Notepad ≠ notepad)

Et Linux **ne reconnaît pas les extensions (.exe, .bat, etc.)** — il exécute directement les fichiers marqués comme *exécutables* (chmod +x).

Que se passe-t-il si l'on tape « notepad » ?

Quand tu tapes :

notepad

➡ Le Bloc-notes Windows s'ouvre. ✨

Pourquoi ?

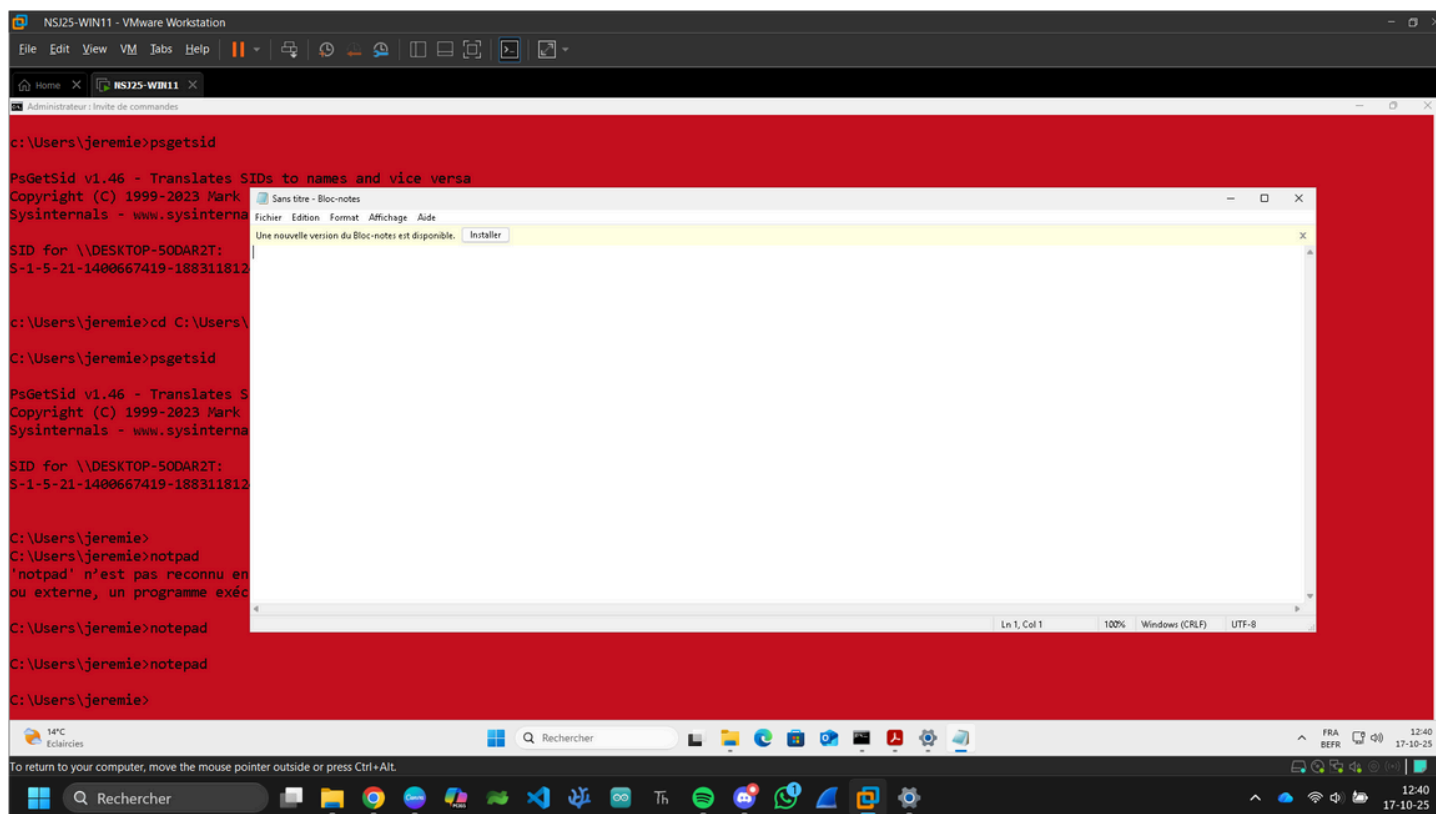
Parce que **notepad.exe** est un **programme système** situé dans :

C:\Windows\System32

Et ce dossier est **déjà dans le PATH** par défaut.

Donc Windows trouve automatiquement notepad.exe sans que tu aies à taper son chemin complet :

C:\Windows\System32\notepad.exe



Pourquoi cela fonctionne-t-il ?

Parce que :

- Windows cherche **dans tous les dossiers listés dans la variable PATH** ;
- Il trouve **le fichier exécutable correspondant** (ici notepad.exe) ;
- Il **exécute ce fichier**.

Autrement dit :

- ➔ Tu n'as pas besoin d'écrire le chemin complet vers notepad.exe
- ➔ Il suffit que le dossier contenant ce programme soit **dans le PATH**.

Cela fonctionne-t-il pour toutes les extensions ?

Pas tout à fait.

Windows a une autre variable d'environnement appelée **PATHTEXT**, qui définit **les extensions reconnues comme exécutables** sans qu'on les tape.

Exemple de PATHTEXT :

.COM;.EXE;.BAT;.CMD;.VBS;.JS;.PY;.MSC

- ➔ Donc quand tu tapes notepad, Windows teste :

1. notepad.com
 2. notepad.exe ✓
 3. notepad.bat
 4. notepad.cmd
- ... dans tous les dossiers du PATH.

Si un de ces fichiers existe, il l'exécute.

⚠ Donc :

- notepad fonctionne ✓ (car c'est un .exe)
- script.bat fonctionne aussi ✓ (si dans le PATH)
- readme.txt ✗ ne s'ouvre pas (ce n'est pas une extension exécutable)
- programme.py fonctionne **seulement si Python est installé et associé à .py** dans PATHTEXT ou via l'association de fichiers.

The screenshot shows a presentation slide titled "Principes de sécurité" (Principles of security). The slide content is as follows:

principal de sécurité, c'est-à-dire chaque utilisateur, groupe, ordinateur ou compte système, au sein d'un environnement Windows.

Un SID est une chaîne alphanumérique structurée, par exemple :

```
PS C:\Users\Laurent> psgetsid
```

SID for \\DESKTOP-V2QJ16O:
S-1-5-21-3558150510-1641510168-375335728

Le SID correspond à quel objet ?

Essayer de déterminer le rôle de chaque partie du SID.

```
PS C:\Users\Laurent> psgetsid $env:USERNAME
```

SID for \\DESKTOP-V2QJ16O:
S-1-5-21-3558150510-1641510168-375335728-1001

Que signifie le fait que les 3 champs avant le RID soit identique ?

Similitude avec un autre OS ?

Qu'est-ce qu'un principal de sécurité ?

Un **principal de sécurité** (*security principal*) est **tout élément auquel Windows peut attribuer des autorisations (droits d'accès, permissions, etc.)**.

Cela inclut :

- 👤 les **utilisateurs** (ex. : toi, "Laurent", "Administrateur")
- 👤 les **groupes** (ex. : "Administrateurs", "Utilisateurs")
- 💻 les **ordinateurs**

-  les **comptes système** (ex. : SYSTEM, NETWORK SERVICE, etc.)

Chaque principal est identifié **de façon unique** par un **SID (Security Identifier)**.

Exemple de SID affiché :

S-1-5-21-3558150510-1641510168-375335728-1001

C'est une chaîne **structurée** : chaque partie a une signification.

Structure détaillée d'un SID

Élément	Exemple	Signification
S	S	Indique qu'il s'agit d'un Security Identifier
1	1	Version du format SID (toujours 1 actuellement)
5	5	Autorité d'identification (ici 5 = NT Authority)
21	21	Indique que le SID est généré pour un domaine ou un ordinateur local
3558150510-1641510168-375335728	ces trois nombres	Identifiant unique du domaine ou de l'ordinateur local
1001	le dernier nombre	C'est le RID (Relative Identifier) → identifie le compte utilisateur dans ce domaine ou machine

Le rôle du SID affiché

Quand tu exécutes :

psgetsid

➡ Tu obtiens le **SID de l'ordinateur local** :

S-1-5-21-3558150510-1641510168-375335728

la Ce SID identifie **la machine Windows** (le poste de travail)a.

Quand tu exécutes :

psgetsid \$env:USERNAME

➡ Tu obtiens le **SID de ton compte utilisateur**, par exemple :

S-1-5-21-3558150510-1641510168-375335728-1001

Ce SID identifie **ton utilisateur "Laurent"**.

Que signifient les 3 champs identiques avant le RID ?

Ce sont :

3558150510-1641510168-375335728

➡ Ils sont **identiques** pour le SID de la machine **et** pour le SID de l'utilisateur.

Cela veut dire que :

Les deux appartiennent **au même domaine ou ordinateur local**.

En résumé :

- Ces trois champs = identifiant du **domaine de sécurité** (ou machine locale)
- Le dernier champ (RID) = identifiant **unique du compte dans ce domaine**

Ainsi :

SID	Appartenance	Objet
S-1-5-21-3558150510-1641510168-375335728	Domaine local	Machine
S-1-5-21-3558150510-1641510168-375335728-1001	Même domaine	Compte utilisateur "Laurent"

Similitude avec un autre OS ?

Oui, **Linux / Unix** utilise un principe **presque identique** avec :

- un **UID (User ID)** pour chaque utilisateur
- un **GID (Group ID)** pour chaque groupe

Par exemple sous **Linux** :

id jeremie

donne :

```
uid=1001(jeremie) gid=100(users) groups=100(users),27(sudo)
```

➔ Le **UID (1001)** joue un rôle **équivalent au RID du SID**.

Chaque utilisateur a un identifiant **unique** au sein du système ou du domaine.

Les RID 500 et 501 sont réservés. Retrouver leur propriétaire.

Rappel : qu'est-ce que le RID ?

Le **RID** (*Relative Identifier*) est **le dernier nombre du SID**.

Il **identifie chaque compte de manière unique** dans le domaine ou sur l'ordinateur local.

Exemple :

```
S-1-5-21-3558150510-1641510168-375335728-1001
```

➔ RID = **1001**

➔ C'est l'utilisateur local (par exemple "Laurent")

RID spéciaux : 500 et 501

Certains RID sont **réservés par Windows** pour des comptes **système prédéfinis**.

RID	Compte associé	Description
500	Administrateur	Compte Administrateur local intégré (créé automatiquement à l'installation). Toujours présent, même s'il est renommé. Peut tout faire sur le système.
501	Invité (Guest)	Compte Guest intégré. Très limité, souvent désactivé par défaut.

Ces deux comptes existent **même si on change leur nom**.

Ce sont leurs **SIDs** qui déterminent réellement leurs identités, pas leur libellé.

Vérification avec PowerShell

Tu peux les retrouver avec :

```
Get-LocalUser | Select Name, SID
```

Tu obtiendras quelque chose comme :

Name	SID
Administrateur	S-1-5-21-3558150510-1641510168-375335728-500
Invité	S-1-5-21-3558150510-1641510168-375335728-501
Laurent	S-1-5-21-3558150510-1641510168-375335728-1001

➔ On voit bien que les comptes "Administrateur" et "Invité" ont toujours les RIDs **500** et **501**.

Le compte utilisé lors de l'installation d'une application à un SID qui déroge à la règle.

Explication :

Quand tu installes une application, Windows stocke dans la base de registre **les droits d'accès et les propriétaires** à partir du **SID**, pas du nom de l'utilisateur.

Donc si tu installes un programme avec un **compte administrateur local**, il sera lié au SID de ce compte.

➡ Par exemple :

S-1-5-21-3558150510-1641510168-375335728-1001

C'est ce SID qui aura les **droits d'installation et de modification** sur les fichiers ou clés de registre du programme.

Que ce passe-t-il, au niveau des droits d'accès si l'on renomme "AdminLocal" en "AdminPC" ?

Quand tu renommes un utilisateur (ex. via Panneau de configuration ou Rename-LocalUser), **le SID ne change pas**.

Exemple :

Avant :

Nom : AdminLocal

SID : S-1-5-21-3558150510-1641510168-375335728-1005

Après :

Nom : AdminPC

SID : S-1-5-21-3558150510-1641510168-375335728-1005

Le **nom visible** change

Le **SID** reste identique

Conséquence :

Les **droits d'accès (ACL)**, les **propriétaires de fichiers**, les **autorisations de registre**, etc., sont **associés au SID**, pas au nom.

Même après le renommage, **les droits d'accès restent exactement les mêmes**.
Windows continue de reconnaître ce compte comme le même principal de sécurité.

! **Exception :**

Si tu **supprimes** un compte, puis **en recrées un autre avec le même nom**,

➔ le nouveau aura **un autre SID**.

➔ Tous les droits de l'ancien seront **perdus**, car ils pointaient vers l'ancien SID.

The screenshot displays the Windows 'Utilisateurs et groupes' (Users and Groups) window. The 'Groups' folder is expanded, showing a list of system groups. The 'Administrators' group is highlighted. A green callout box asks 'Quel est le rôle de ses groupes ?' (What is the role of these groups?). Another green callout box asks 'Quel est le SID du groupe \"Administrateurs\" ?' (What is the SID of the 'Administrators' group?). The background shows a web browser with a Moodle page titled 'Support de cours de Laurent Schalkwijk'.

Rôles des groupes

Groupe	Rôle / Droits
Administrators	Contrôle total sur l'ordinateur ou le domaine. Peut installer des logiciels, changer les paramètres système, gérer les comptes et droits d'accès.
Backup Operators	Peut sauvegarder et restaurer des fichiers, même ceux auxquels il n'a pas accès normalement.
Event Log Readers	Peut lire les journaux d'événements Windows, utile pour la supervision et le diagnostic.
Guests (Invités)	Compte limité pour les utilisateurs temporaires. Accès très restreint au système, pas de modification de paramètres système.
OpenSSH Users	Droit de se connecter via SSH (si le serveur OpenSSH est installé).
Power Users	Peut effectuer certaines tâches administratives limitées mais ne contrôle pas totalement le système (ex: installer certaines applications, créer des comptes limités).
Remote Desktop Users	Droit de se connecter au PC via le Bureau à distance (RDP).
Users (Utilisateurs)	Compte standard. Peut utiliser le système et installer certaines applications, mais ne peut pas changer les paramètres système globaux ni gérer les autres comptes.

SID du groupe "Administrateurs"

SID par défaut : **S-1-5-32-544**

- **S-1-5-32** → indique un groupe intégré local
- **544** → identifiant relatif du groupe Administrateurs

Quels sont les principaux objets sécurisés sous Windows ?

Fichiers et dossiers

Type d'objet : NTFS (système de fichiers Windows)

Sécurité : permissions ACL définissent qui peut lire, écrire, exécuter ou modifier un fichier/dossier.

Exemple : C:\Windows\System32\config\SAM est protégé.

Clés de registre

Type d'objet : clés et valeurs du registre Windows

Sécurité : ACL sur les clés du registre (HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, etc.)

Exemple : les clés de configuration système sensibles (ex: HKEY_LOCAL_MACHINE\SAM)

Objets partagés réseau

Type d'objet : dossiers partagés ou imprimantes sur le réseau

Sécurité : contrôle d'accès partagé (Share Permissions) et NTFS ACL

Exemple : \\Serveur\Partage

Processus

Type d'objet : chaque processus en cours d'exécution

Sécurité : chaque processus a un **token d'accès** qui définit les droits de son utilisateur et son intégrité

Exemple : un processus lancé par un utilisateur standard ne peut pas tuer un processus système

Threads

Type d'objet : unité d'exécution dans un processus

Sécurité : contrôle d'accès pour suspendre, terminer ou modifier un thread

Services Windows

Type d'objet : services installés

Sécurité : ACL pour démarrer, arrêter, configurer le service

Exemple : service WinDefend (Windows Defender)

Mutex, sémaphores et autres objets de synchronisation

Type d'objet : objets kernel pour la synchronisation entre processus

Sécurité : ACL pour permettre ou interdire l'accès

Exemple : Global\MyMutex

Pipes nommés (Named Pipes)

Type d'objet : canal de communication inter-processus

Sécurité : ACL définissant qui peut lire ou écrire sur le pipe

Exemple : \\.\pipe\spoolss

Sockets (WinSock)

Pas directement sécurisés via ACL, mais souvent contrôlés par les règles du firewall Windows et les privilèges d'utilisateur

Objets périphériques

Type d'objet : imprimantes, lecteurs, périphériques matériels

Sécurité : ACL qui contrôlent qui peut lire, écrire ou configurer le périphérique

Résumé :

Windows applique la sécurité via **ACL et tokens d'accès** sur des objets variés : fichiers, dossiers, clés de registre, processus, services, objets kernel, pipes, partages et périphériques.

Après avoir fermé toutes les applications:

Ouvrer dans un terminal un onglet "Powershell" et un "Command Prompt";

Ouvrer Firefox;

Ouvrer l'application "Process Explorer 64 bits" de la suite SysInternals. En explorant le processus "Explorer.exe" ainsi que les ceux correspondants aux applications ouvertes, que pouvez-vous dire des droits ?

("Properties" -> "Security tab") La commande "whoami" avec les paramètres (/USER - /GROUPS et /PRIV) peut vous aider à comprendre...

Préparation

Tu as ouvert :

1. PowerShell
2. Command Prompt (cmd.exe)

- 3. Firefox
- 4. Process Explorer 64 bits

Tu vas explorer :

- Explorer.exe (le processus du shell Windows)
- Les processus des applications ouvertes (PowerShell, cmd, Firefox)

Observation des droits via Process Explorer

Étapes :

Dans **Process Explorer**, clique droit sur un processus → **Properties** → onglet **Security**.

Tu verras plusieurs informations :

- **Owner (Propriétaire)** : utilisateur qui a lancé le processus.
- **Group** : groupe principal de l'utilisateur.
- **Access Rights** : droits sur le processus (Full Control, Read, Write, etc.).

Ce que tu constateras :

Processus	Propriétaire	Droits / Observations
Explorer.exe	Ton utilisateur (ex: Jeremie)	Full Control sur le processus lui-même. Peut gérer les fenêtres, charger des DLL, lancer d'autres applications.
PowerShell / cmd.exe	Ton utilisateur	Full Control pour l'utilisateur. Droits limités pour les autres comptes (Users, Guests).
Firefox	Ton utilisateur	Processus sandboxé : droits complets pour l'utilisateur courant mais droits restreints sur certaines ressources système (ex: registre, dossiers système).

Remarque :

Les processus héritent du **token d'accès** de l'utilisateur qui les lance.

Les **processus système** (ex: svchost.exe) peuvent être lancés par **SYSTEM** ou **TrustedInstaller**, donc ils ont beaucoup plus de droits que ton utilisateur standard.

Vérification via la commande whoami

Dans un terminal, tu peux exécuter :

whoami /user

whoami /groups

whoami /priv

/user → affiche ton SID utilisateur actuel.

/groups → affiche les groupes dont tu fais partie et tes SID relatifs (Administrators, Users, etc.).

/priv → affiche les **privileges du token**, par exemple :

- SeDebugPrivilege → permet de déboguer tous les processus.
- SeShutdownPrivilege → permet d'éteindre le système.
- La plupart sont **désactivés par défaut**, mais peuvent être activés si nécessaire.

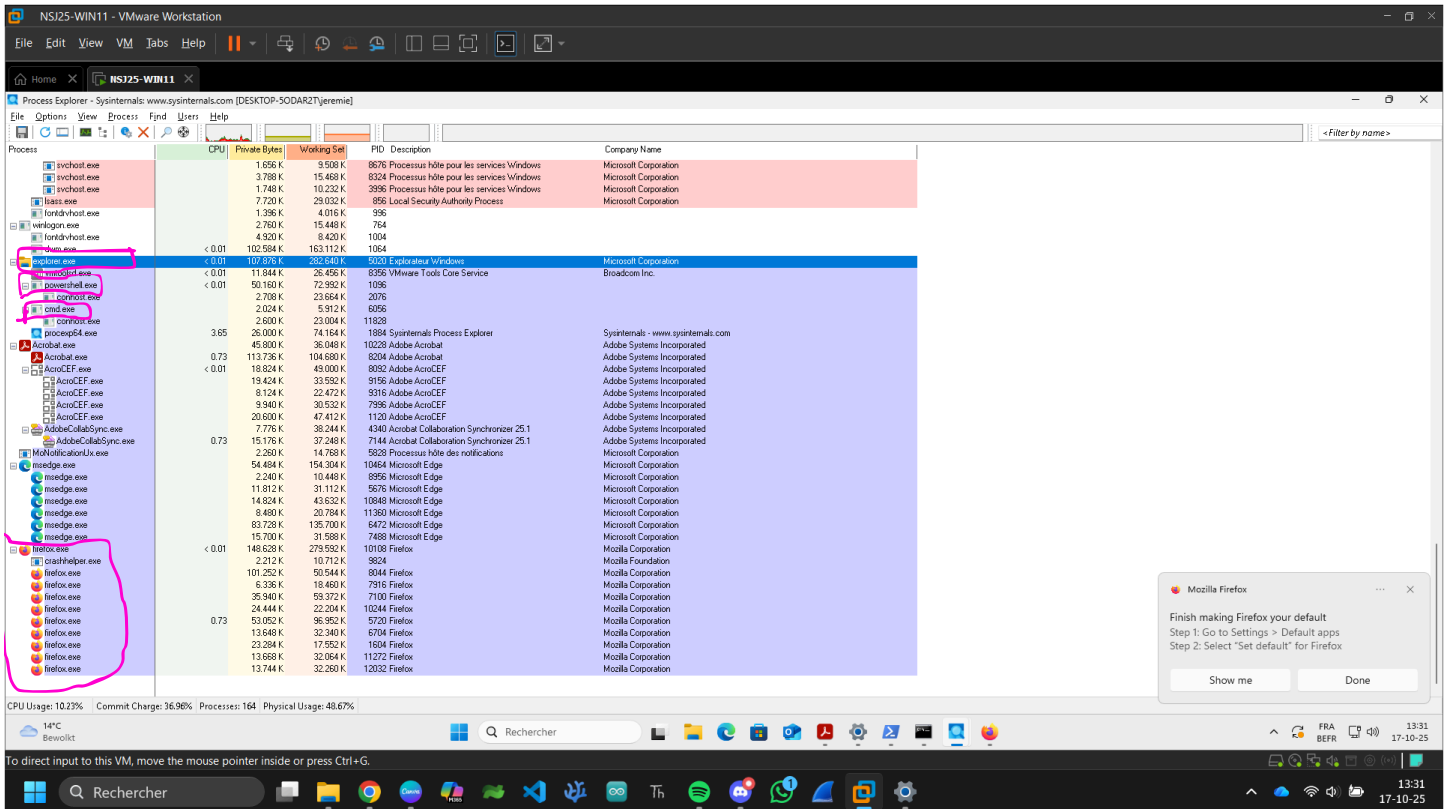
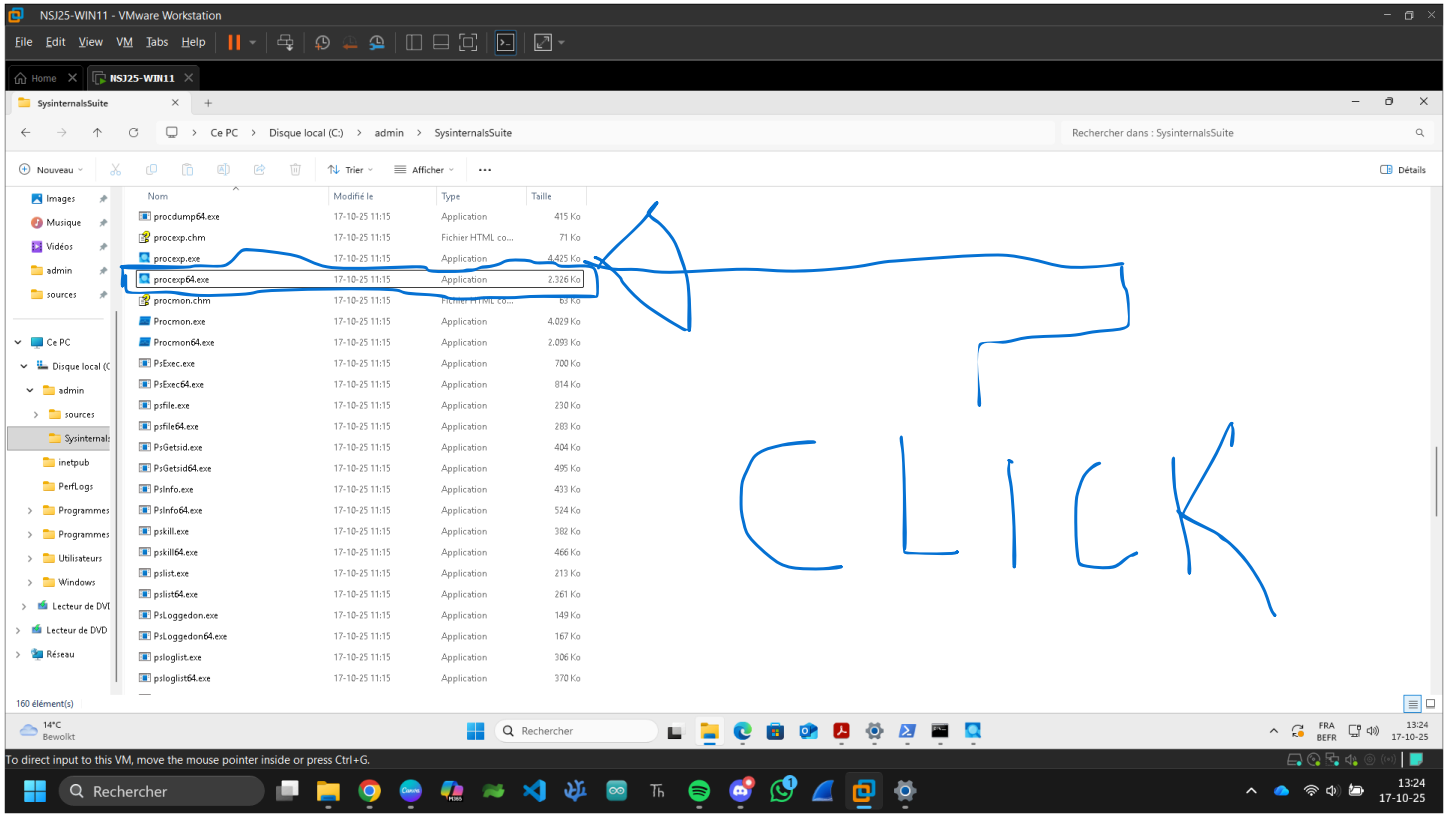
Exemple concret :

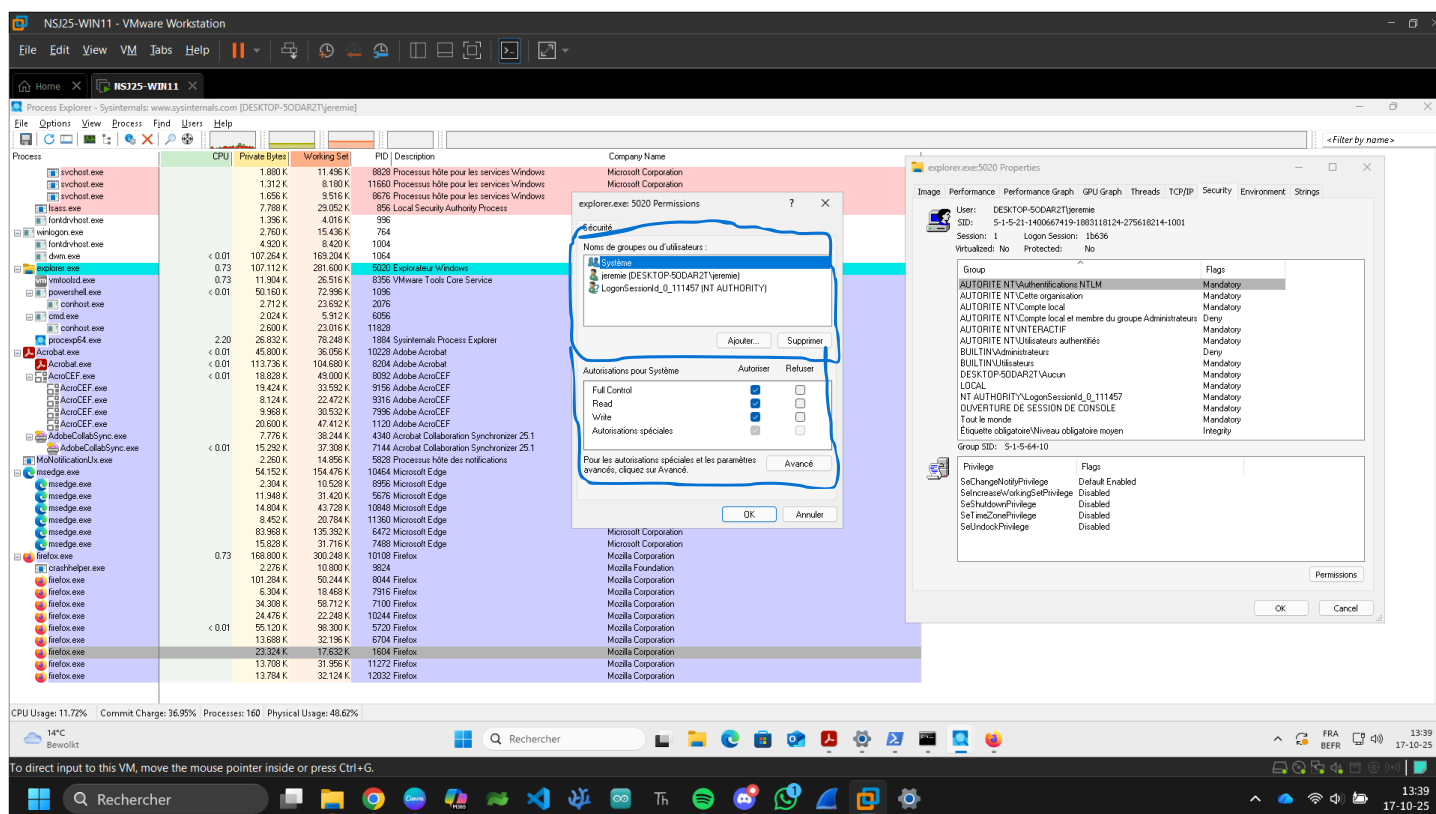
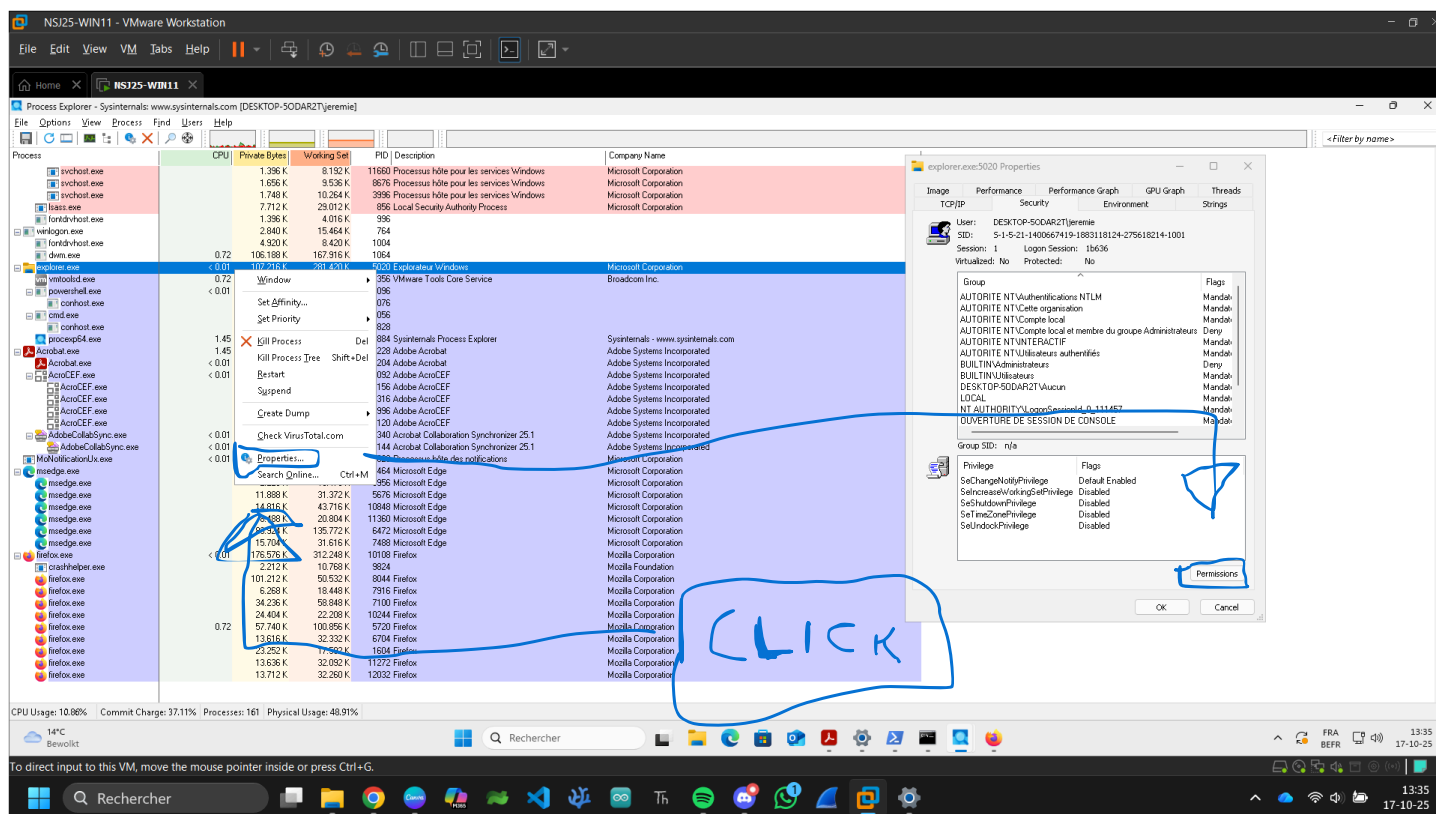
Si ton utilisateur fait partie du groupe **Administrators** :

- Explorer.exe et PowerShell auront le droit d'interagir avec beaucoup d'objets système.
- Les applications standards comme Firefox auront seulement accès aux fichiers et dossiers de ton profil utilisateur.

Synthèse

1. **Chaque processus a un token d'accès** qui définit ses droits.
2. **Le propriétaire** du processus est généralement l'utilisateur qui l'a lancé.
3. **Les groupes et privilèges** déterminent ce que le processus peut faire sur le système et sur les autres processus.
4. Les **processus système** ont beaucoup plus de privilèges que les processus utilisateurs.
5. **whoami /priv + Process Explorer** → te montre exactement quels privilèges sont actifs et quels groupes sont associés au processus.





Deux sections principales dans "Permissions"

Nom du groupe ou de l'utilisateur (en haut)

Cette section liste **tous les utilisateurs et groupes qui ont des permissions sur le processus.**

Exemple typique :

- **SYSTEM** → le système Windows
- **Ton utilisateur** → toi, Jeremie
- **Administrators / Users / Logon / etc.** → groupes dont fait partie le processus

Chaque entrée ici va avoir ses **droits propres** sur l'objet.

Permissions (en bas)

Quand tu sélectionnes un utilisateur ou un groupe dans la section du dessus, la section **Permissions** te montre :

Colonne	Signification
Full Control	Tout est autorisé : lire, écrire, terminer le processus, changer les droits, etc.
Read	Lire les informations sur le processus (nom, ID, modules, threads...)
Write / Écriture	Modifier le processus : changer son état, terminer un thread, injecter une DLL...
Special Permissions / Autorisations spéciales	Droits précis plus fins, comme modifier le token d'accès, suspendre un thread, ou changer les droits sur le processus.
Allow / Refuse (Autoriser / Refuser)	Détermine si le droit est accordé ou explicitement refusé.

Important : **Refuser** a toujours priorité sur **Autoriser**.

Exemple concret

Si tu sélectionnes **SYSTEM** :

- Full Control → coché → le système peut faire absolument tout sur ce processus.

Si tu sélectionnes **ton compte utilisateur (Jeremie)** :

- Read → coché → tu peux voir les infos du processus

- Write → parfois coché → tu peux interagir un peu avec le processus
- Full Control → parfois non coché → tu ne peux pas terminer tous les processus système

Si tu sélectionnes un groupe comme **Users** :

- Souvent seulement Read est autorisé, Write et Full Control sont désactivés.

Comment lire correctement

1. Clique sur un utilisateur ou un groupe.
2. Regarde quelles cases sont cochées dans la partie Permissions.
3. **Autoriser** = droit actif pour ce groupe ou utilisateur.
4. **Refuser** = droit bloqué même si un autre groupe l'autorise.